

Experiment - 1

Aim: - Write down the problem statement for the E- Learning Portal.

1. Problem Statement:

Current education systems are hampered by inefficiencies in accessibility, resource distribution, and student engagement. Traditional classroom methods often fail to adapt to dynamic conditions, resulting in limited reach, inconsistent learning outcomes, and difficulties in monitoring student progress. Study materials are often scattered across multiple sources, making access inconvenient and unorganized. Additionally, rigid teaching methods do not support personalized learning. To address these issues, there is a need for an advanced E-Learning Portal that integrates real-time content delivery, employs adaptive learning models, and provides interactive features to optimize education quality and enhance the overall learning experience.

2. Background:

Education is a cornerstone of personal and professional development, facilitating the growth of millions of learners worldwide. Effective management of learning resources is crucial for ensuring accessibility, efficiency, and engagement. However, the complexity of modern education—such as course management, real-time content delivery, and performance monitoring—presents significant challenges. Traditional learning systems often fall short in addressing these issues, leading to reduced efficiency, increased workload for instructors, and limited student engagement. This highlights the need for a centralized and interactive E-Learning platform that overcomes these limitations.

3. Current Challenges:

- **Limited Accessibility:** Students often face barriers to accessing study materials outside physical classrooms.
- **Scattered Resources:** Notes, videos, and assignments are distributed across different platforms, causing confusion.
- **Lack of Real-Time Interaction:** Many existing systems do not support effective real-time communication between students and teachers.
- **Rigid Learning Patterns:** Traditional methods do not allow personalized or adaptive learning approaches.

- **Performance Tracking Issues:** Teachers find it difficult to monitor student progress and provide timely feedback.

4. Existing system problem:

- **Fragmented Platforms:** Current systems rely on multiple disconnected tools (e.g., separate apps for lectures, quizzes, and assignments).
- **Limited Data Utilization:** Most systems do not leverage analytics to understand student learning patterns or identify weak areas.
- **Low Engagement:** Absence of interactive elements (like discussion forums, live quizzes, feedback loops) results in passive learning.

5. Objectives:

To address these challenges, the objective is to design and implement an E-Learning Portal that enhances accessibility, engagement, and performance monitoring. The system should integrate all course-related resources, facilitate interactive learning, and provide personalized feedback to improve overall educational outcomes.

6. Proposed Goals:

- **Centralized Resource Management:** Create a single platform for courses, notes, videos, assignments, and tests.
- **Interactive Learning:** Implement features like discussion forums, live quizzes, and feedback systems to boost engagement.
- **Adaptive Learning:** Provide personalized learning paths based on student performance and participation.
- **Performance Analytics:** Deploy tools to generate progress reports, helping both teachers and students track learning outcomes.
- **Teacher Support Tools:** Enable instructors to upload content, evaluate assignments, and automate grading.
- **Scalable Platform:** Design the portal to support a large number of students and courses efficiently.

7. Expected Outcomes:

- **Enhanced Accessibility:** Students can access learning materials anytime, anywhere, leading to improved flexibility.
- **Improved Engagement:** Interactive and real-time learning tools will keep students more actively involved.
- **Optimized Performance Tracking:** Teachers will be able to monitor student progress and provide timely interventions.
- **Personalized Learning:** Adaptive features will allow learners to study at their own pace, improving overall outcomes.
- **Centralized System:** A unified platform will reduce confusion and streamline the learning process.

8. Requirements:

Hardware Requirements –

➤ Server/Host Machine

- Processor: Minimum Intel i3 / AMD equivalent or higher
- RAM: 4 GB or more
- Storage: 250 GB minimum (HDD/SSD)
- Network: Ethernet/Wi-Fi support for LAN or internet connectivity

➤ Client Machines

- Processor: Minimum Intel Pentium Dual Core or higher
- RAM: 2 GB or more
- Storage: 50 GB free space
- Input Devices: Keyboard, Mouse/Touchpad
- Output Devices: Monitor (minimum 1024×768 resolution), Printer (for receipts/reports, optional)

Software Requirements –

- Operating System: Windows / Linux / macOS (any modern version)
- Database Management System (DBMS): MySQL / PostgreSQL / Oracle / SQL Server

- Programming Language & Frameworks: Java / Python / PHP / .NET (depending on project design)
- Web Server (if web-based): Apache / Nginx / IIS
- Browser Support (if web-based): Chrome, Firefox, Edge
- Development Environment: Eclipse / VS Code / NetBeans / PyCharm
- Backup and Security Tools: Database backup utility, antivirus, and firewall

9. Conclusion:

By addressing challenges of accessibility, engagement, and resource management, the proposed E-Learning Portal aims to transform the learning process. This system will not only improve efficiency and reduce administrative effort but also enhance student participation, performance, and satisfaction. Ultimately, it provides a modern, scalable, and reliable educational infrastructure that bridges the gap between traditional and digital learning.

Software Requirements Specification

for

< E-Learning Portal >

Version 1.0 approved

Prepared by Kunal

Maharaja Agrasen Institute of Technology

<10/Sept/2025>

Table of Contents

1.	Introduction	3
1.1	Purpose	3
1.2	Document Conventions	3
1.3	Intended Audience and Reading Suggestions	3
1.4	Product Scope	4
1.5	References	4
2.	Overall Description	5
2.1	Product Perspective	5
2.2	Product Functions	5
2.3	User Classes and Characteristics	5
2.4	Operating Environment	6
2.5	Design and Implementation Constraints	6
2.6	User Documentation	6
2.7	Assumptions and Dependencies	6
3.	External Interface Requirements	7
3.1	User Interfaces	7
3.2	Hardware Interfaces	7
3.3	Software Interfaces	7
3.4	Communications Interfaces	7
4.	System Features	7
4.1	System Feature 1	8
4.2	System Feature 2 (and so on)	8
5.	Other Nonfunctional Requirements	9
5.1	Performance Requirements	9
5.2	Safety Requirements	9
5.3	Security Requirements	9
5.4	Software Quality Attributes	9
5.5	Business Rules	9
6.	Other Requirements	10

Revision History

Name	Date	Reason For Changes	Version

EXPERIMENT 2

Aim: -Do requirement analysis and develop Software Requirement Specification Sheet (SRS) for the E- Learning Portal.

1. Introduction

1.1 Purpose:

The purpose of this document is to specify the detailed software requirements for the E-Learning Portal, an online education management system. The portal aims to provide students and teachers with a centralized, interactive, and secure platform for managing academic courses, learning resources, assessments, communication, and performance evaluation.

This document serves as:

- A contract between the stakeholders (management, teachers, students, developers).
- A blueprint for developers and testers to design, implement, and validate the system.
- A reference for administrators to understand how the system will streamline operations.

1.2 Document Conventions:

- Headings and subheadings follow the IEEE 830-1998 SRS standard.
- Functional requirements are grouped by modules (Student, Teacher, Admin).
- Non-functional requirements are grouped in Section 5.
- Diagrams (DFD, Use Case, ER) will follow UML notation.

1.3 Intended Audience and Reading Suggestions:

This document is intended for:

- **Project Managers** – For planning, monitoring, and resource allocation.
- **Developers/Designers** – For implementation and database design.
- **Testers** – For preparing test cases and validating requirements.
- **Administrators (Teachers & Institute staff)** – For understanding system capabilities.
- **Students** – As end-users benefiting from the platform.

Recommendation:

- Project managers should focus on Sections 1 & 2.
- Developers should focus on Sections 3, 4, and Appendix B.
- Testers should focus on Sections 4 & 5.

1.4 Product Scope:

The E-Learning Portal is designed to enhance learning outcomes by providing an interactive, accessible, and scalable digital education platform. It will allow:

- **Students** to register, enroll in courses, access study materials, participate in quizzes, submit assignments, and view results.
- **Teachers** to create courses, upload learning content, manage assessments, track student performance, and provide feedback.
- **Administrators** to monitor system usage, manage users, generate reports, and maintain security.

The project scope includes:

- A web-based system accessible through browsers.
- Separate dashboards for students, teachers, and admins.
- A secure authentication system with role-based access.
- Scalability to support increasing number of users and courses.

1.5 References

- IEEE Std 830-1998 for Software Requirement Specifications.
- Karl E. Wiegers, *Software Requirements*.
- Learning Management Systems (LMS) like Moodle, Coursera, Udemy for feature benchmarking.
- PHP and MySQL Web Development, Luke Welling.

2. Overall Description

2.1 Product Perspective:

The E-Learning Portal is a standalone web application but can be extended into a cloud-based or mobile-integrated system in the future. It will replace fragmented tools (Google Classroom, WhatsApp groups, PDFs shared separately) with a unified learning environment.

It includes:

- **Front-End Interface:** Web-based, responsive UI for students, teachers, and admins.
- **Back-End System:** Handles authentication, course management, and performance tracking.
- **Database:** Stores student records, courses, study materials, grades, and reports.

2.2 Product Functions:

The key functions of the portal include:

- **Student Services:** Registration, course enrollment, study material access, quizzes, assignments, grades.
- **Teacher Services:** Course creation, content upload, quiz and assignment management, grading.
- **Admin Services:** User management, reporting, content approval, system monitoring.
- **General Features:** Notifications, discussion forums, and role-based dashboards.

2.3 User Classes and Characteristics

- **Students:**
 - Knowledge Level: Basic computer literacy.
 - Access: Course enrollment, materials, quizzes, results.
- **Teachers:**
 - Knowledge Level: Intermediate IT knowledge.
 - Access: Upload and manage content, evaluate students.
- **Administrators:**

- Knowledge Level: Advanced technical and management knowledge.
- Access: User accounts, system logs, and performance analytics.

2.4 Operating Environment:

- **Platform:** Windows/Linux.
- **Server:** Apache/Nginx.
- **Database:** MySQL/PostgreSQL.
- **Languages:** PHP, JavaScript, HTML, CSS.
- **Browser Support:** Chrome, Firefox, Edge, Safari.
- **Devices:** Desktop, Laptop, Tablets, Smartphones.

2.5 Design and Implementation Constraints:

- Must support role-based access control.
- Data security via encryption (AES, SHA-256).
- Developed using open-source technologies (PHP, MySQL).
- Should handle at least 500 concurrent users.

2.6 User Documentation:

- **Student Guide:** Course enrollment, accessing content.
- **Teacher Guide:** Course creation, uploading content, grading.
- **Admin Guide:** User management, system setup, troubleshooting.

2.7 Assumptions and Dependencies:

- Users have stable internet access.
- Hosting server supports PHP and MySQL.
- Teachers/students are familiar with online platforms.

3. External Interface Requirements

3.1 User Interfaces:

- **Login Page:** User authentication (Student/Teacher/Admin).
- **Student Dashboard:** Courses, materials, assignments, grades.
- **Teacher Dashboard:** Course management, content upload, grading.
- **Admin Dashboard:** User management, system monitoring.
- **Discussion Forum:** Student-teacher interactions.

3.2 Hardware Interfaces:

- **Client Devices:** Any standard PC or mobile device with 2 GB RAM, internet-enabled.
- **Server:** Minimum 8 GB RAM, Quad-core processor, 100 GB storage.

3.3 Software Interfaces:

- **Database:** MySQL/PostgreSQL.
- **Backend:** PHP/Node.js.
- **Frontend:** HTML, CSS, JavaScript.
- **Communication:** REST APIs.

3.4 Communication Interfaces:

- **HTTP/HTTPS protocols.**
- **Email & SMS notifications** for important updates.
- **Push notifications** (future mobile app integration).

4. System Features

4.1 User Module:

- Secure registration/login with authentication.
- Forgot password and password reset functionality.

4.2 Student Module:

- Profile management.
- Enroll in courses.
- Access study materials (videos, notes, PDFs).
- Attempt quizzes and submit assignments.
- Track progress and grades.

4.3 Teacher Module:

- Create and manage courses.
- Upload materials (video, notes, PPTs).
- Create quizzes and assignments.
- Evaluate submissions and provide feedback.

4.4 Admin Module:

- Add/remove/approve users.
- Manage courses and categories.
- Generate reports (student progress, usage statistics).
- Monitor system logs.

4.5 Assessment Module:

- Auto-grading for objective quizzes.
- Manual grading for subjective assignments.
- Instant result generation.

4.6 Report Generation Module:

- Student performance reports.
- Teacher activity reports.

- Admin-level system usage reports.

4.7 Discussion Forum Module:

- Student-teacher Q&A.
- Peer-to-peer discussions.
- Moderation by teachers/admins.

5. Other Non-Functional Requirements

5.1 Performance Requirements:

- Response time < 2 seconds for major operations.
- Handle **500+ concurrent users**.

5.2 Safety Requirements:

- Daily database backup.
- Failover server support.

5.3 Security Requirements:

- **Password encryption** using SHA-256.
- **Role-based access control**.
- **Secure sessions** via HTTPS.

5.4 Software Quality Attributes:

- **Reliability:** 99% uptime.
- **Usability:** User-friendly, responsive design.
- **Maintainability:** Modular, well-documented code.
- **Portability:** Works on multiple OS and browsers.

5.5 Business Rules:

- Only admins can approve course creation.
- Students must enroll before accessing course content.
- Teachers can edit only their own courses.

6. Other Requirements:

- Mobile app integration.
- AI-based adaptive learning recommendations.
- Integration with payment gateways for fee-based courses.

Appendix A: Glossary:

- **LMS:** Learning Management System.
- **Quiz:** Online test within a course.
- **Assignment:** Task/project submitted by students.

Appendix B: Analysis Models:

- **Use Case Diagram** – Student, Teacher, Admin roles.
- **DFD** – Level 0 and Level 1 for course management.
- **ER Diagram** – Students, Courses, Teachers, Assignments entities.

Appendix C: To Be Determined List:

- Exact UI design templates.
- Specific hosting environment.

Experiment 3

Aim: To perform the function-oriented diagram for the E- Learning Portal: Data Flow Diagram (DFD).

Theoretical Foundation of Data Flow Diagrams (DFDs)

A Data Flow Diagram (DFD) is a graphical tool in structured system analysis used to model the flow of data within an information system. It focuses purely on the movement and transformation of data, showing what the system does rather than how it's implemented (which technologies are used). This makes it essential for validating and communicating functional requirements.

Components of Data Flow Diagram (DFD):

DFDs are built using four key symbols (most commonly using the Gane and Sarson notation for clarity):

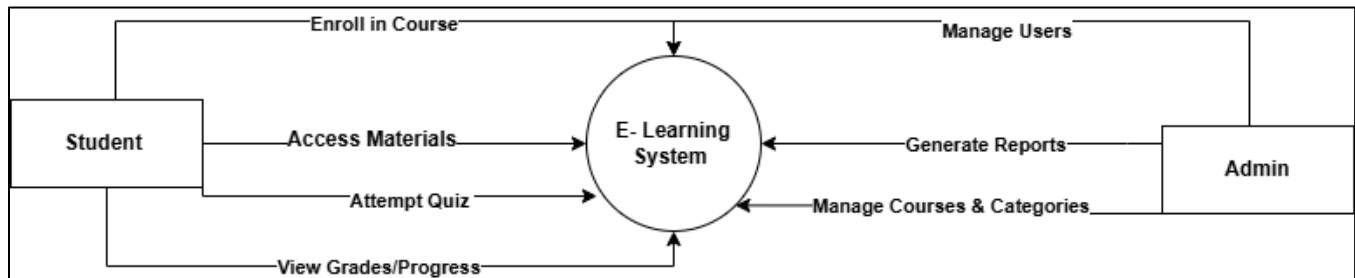
- 1. Process (Circle/Rounded Rectangle):** Represents work being performed that transforms incoming data flow(s) into outgoing data flow(s).
 - Project Example: **2.0 Course & Content Management.**
- 2. External Entity (Rectangle):** A source or destination of data that is outside the boundary of the system being modeled.
 - Project Example: **Student, Teacher, Administrator.**
- 3. Data Flow (Labeled Arrow):** The movement of a packet of information between components. The label defines the specific data being moved.
 - Project Example: **Assessment Submissions, Learning Materials.**
- 4. Data Store (Open-ended Rectangle):** A repository where data is held or stored for later access. This represents internal system files or database tables.
 - Project Example: **D1 User Records, D4 Assessment Data.**

Hierarchical Levels of DFDs (Decomposition)

DFDs use **leveling** to manage system complexity, moving from a general overview to detailed functions.

1. Level 0 DFD (Context Diagram)

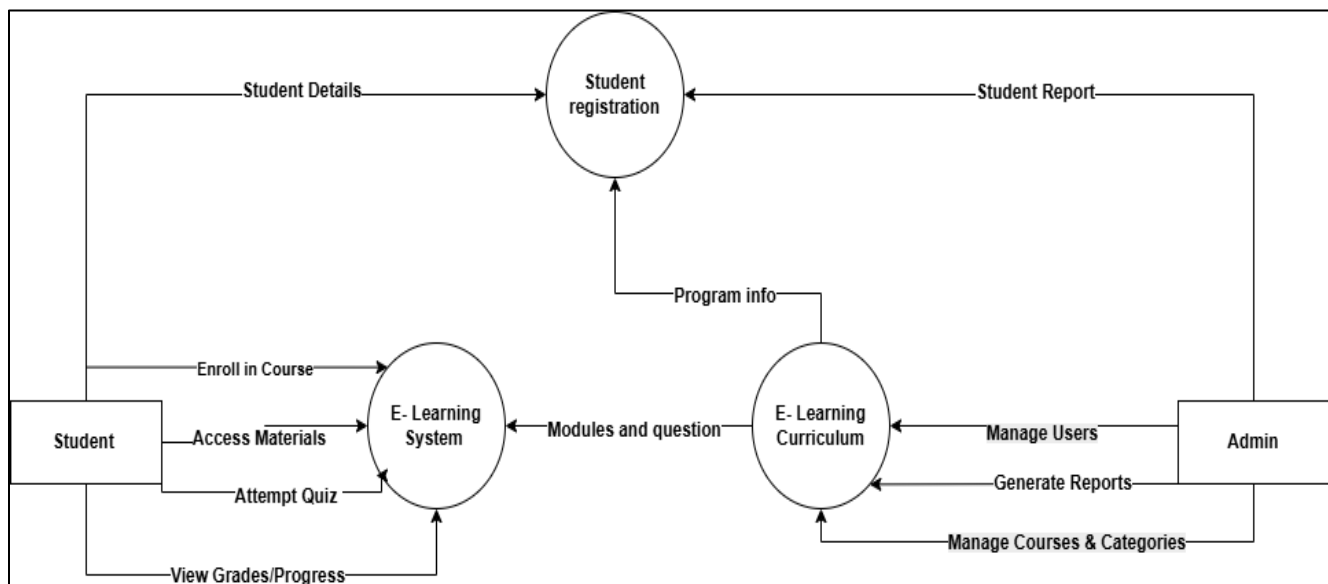
- **Role:** Defines the **entire scope and boundary** of the E-Learning Portal.
- **Structure:** Shows the system as a **single, unified process** (Process 0: E-Learning Portal System).
- **Focus:** Illustrates the primary **external entities** and the major **inputs/outputs** (data flows) that cross the system boundary. It **does not** show internal data stores.



Level-0 Data Flow Diagram (DFD) for E- Learning portal

2. Level 1 DFD

- **Role:** Decomposes the Level 0 process into its **major functional subsystems** or modules.
- **Structure:** Breaks Process 0 into 3-7 core processes, aligning with your SRS modules (e.g., **1.0 User & Access Management, 3.0 Assessment & Grading**).
- **Focus:** Shows how these major processes interact with each other, with external entities, and with the **main internal data stores** (D1, D2, D3, etc.) necessary for system operation.



Level-1 Data Flow Diagram (DFD) for E- Learning portal

Uses of DFDs in the E-Learning Portal Project

1. **Structural Validation:** The leveled approach ensures data balancing—all data flowing into a parent process (like Level 0) must correctly flow in and out of its child processes (Level 1), guaranteeing data integrity and traceability throughout the model.
2. **Bridging Requirements to Design:** The processes defined in the DFDs directly translate into the modules and functions developers must code, while the data stores define the necessary database schema (tables).
3. **Functional Clarity:** They confirm that the system handles critical data paths, such as the flow from a Teacher creating content to that content being stored (D3) and finally retrieved by the Student.
4. **Identification of System Constraints:** By showing data stores and data flows, DFDs help identify areas of heavy data traffic or critical storage needs, which informs the design decisions regarding performance (e.g., optimizing access to the D4 Assessment Data store).

Experiment 4

Aim: Draw the entity relationship diagram for the E-Learning Portal.

Theory and Uses of an ER Diagram for the E-Learning Portal

An Entity Relationship Diagram (ERD) is a structural database model that visually represents the data entities, their attributes, and the relationships between them within the E-Learning Portal system. Its primary function is to capture the complete data requirements and provide a blueprint for database design, focusing on how data is organized and related (the *structure*), not the application logic (the *behaviour*). This makes it essential for database design during the system design phase.

For the E-Learning Portal, this diagram is crucial for modelling how the three key user groups—Students, Teachers, and Administrators—interact with the platform's data entities (User, Course, Enrollment, Grade, etc.) to support the academic process.

Components of the ER Diagram (Project-Specific)

Component	Description	Example Relevant to the E-Learning Portal
Entities	Real-world objects or concepts that contain data	User, Course, Enrollment, Grade, Module, Session, Content, Message
Attributes	Properties or characteristics of entities	user_id, username, email, course_id, title, enrollment_date, grade
Primary Key	Unique identifier for each entity instance	user_id in User entity, course_id in Course entity
Foreign Key	Attribute that creates relationship between entities	instructor_id in Course references user_id in User
Relationships	Associations between entities	User enrolls in Course, Course contains Module, Module has Session
Cardinality	Defines numerical relationship between entities	One-to-Many (1:N), Many-to-Many (M:N)

Uses of an ER Diagram (Project Application)

The Entity Relationship Diagram is a powerful tool specifically utilized for the following functions in the E-Learning Portal project lifecycle:

1. To Define Data Requirements and Structure:

- It visually represents all data entities and their relationships as identified in the system requirements
- Ensures all necessary data for system functionality (user management, course delivery, assessment, communication) is properly modelled

2. To Support Database Implementation:

- Serves as a direct blueprint for creating database tables and relationships in SQL
- Helps in normalizing the database to reduce data redundancy and improve integrity
- Guides the creation of proper indexes and constraints for optimal performance

3. To Clarify Data Relationships and Constraints:

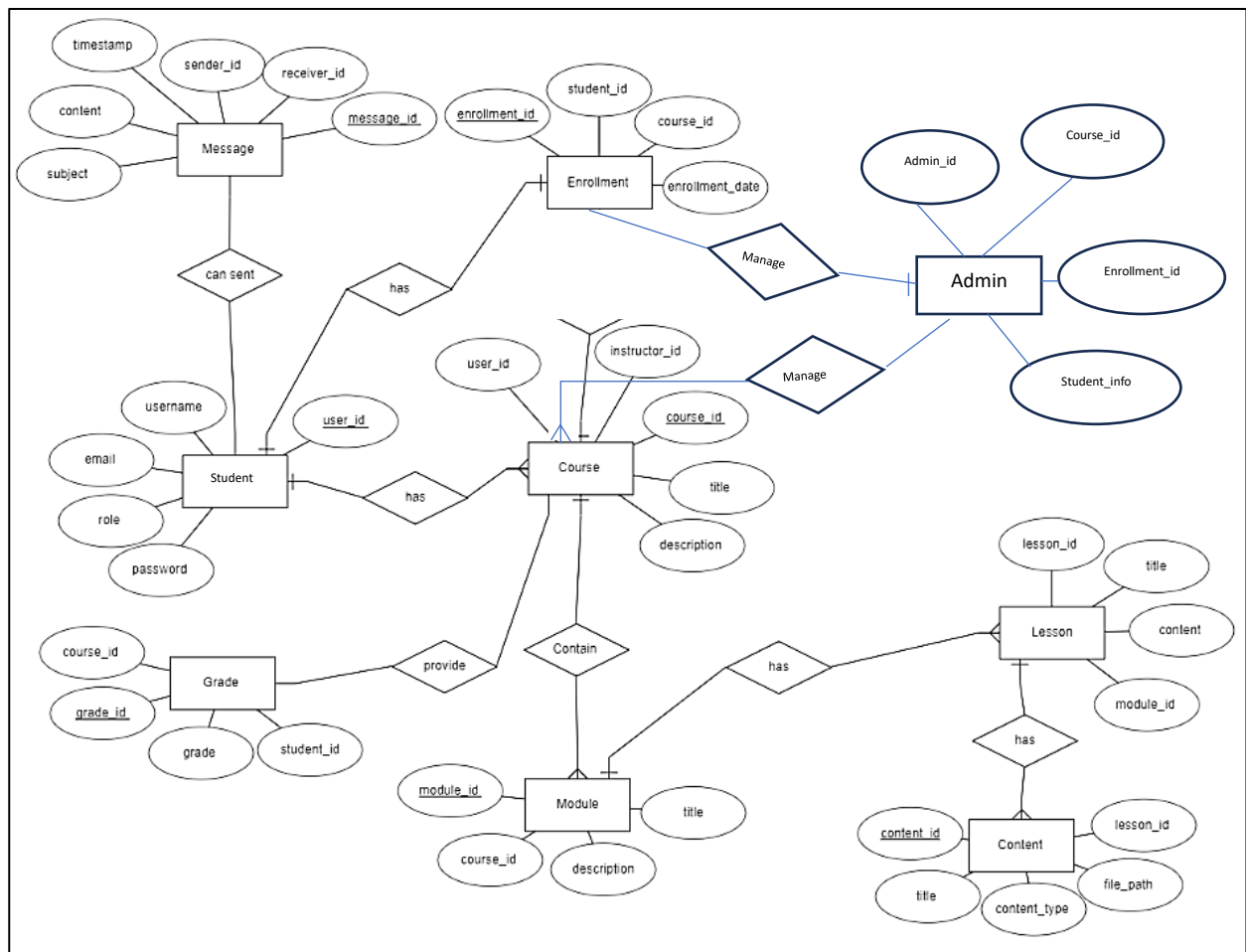
- Clearly shows how different entities relate to each other (e.g., a Student can enroll in multiple Courses, a Course can have multiple Students)
- Defines cardinality constraints (one-to-many, many-to-many relationships)
- Ensures referential integrity through proper foreign key relationships

4. To Facilitate Communication Between Teams:

- Provides a common visual language for developers, database administrators, and stakeholders
- Helps non-technical stakeholders understand how data flows through the system
- Serves as documentation for future maintenance and enhancements

5. To Ensure Data Consistency and Integrity:

- Identifies required attributes and constraints for each entity
- Helps prevent data anomalies through proper normalization
- Ensures that all business rules are captured in the data model



Key Relationships:

1. User - Course Relationship:

- A User (as Instructor) can create multiple Courses (1:N)
- A User (as Student) can enroll in multiple Courses (M:N through Enrollment)

2. Course - Module Relationship:

- A Course can contain multiple Modules (1:N)

3. Module - Session Relationship:

- A Module can have multiple Sessions (1:N)

4. Session - Content Relationship:

- A Session can contain multiple Content items (1:N)

5. User - Grade Relationship:

- A User (Student) can receive multiple Grades (1:N)
- A Course can provide multiple Grades (1:N)

6. User - Message Relationship:

- A User can send multiple Messages (1:N)
- A User can receive multiple Messages (1:N)

Cardinality Constraints:

- **One-to-Many (1:N):** User $\rightarrow$ Course (instructor relationship)
- **One-to-Many (1:N):** Course $\rightarrow$ Module
- **One-to-Many (1:N):** Module $\rightarrow$ Session
- **One-to-Many (1:N):** Session $\rightarrow$ Content
- **One-to-Many (1:N):** User $\rightarrow$ Grade (student perspective)
- **Many-to-Many (M:N):** User $\leftrightarrow$ Course (through Enrollment entity)

Experiment 5

Aim: To perform the function-oriented diagram: Structured chart.

Theory and Uses of an ER Diagram for the Structured chart

A Structure Chart is a systematic, hierarchical diagram used in structured analysis and design methodology (SADM) to represent the organizational architecture of a software system. It serves as a blueprint that illustrates the decomposition of a system into constituent modules and defines the relationships and interactions between them.

Components of the ER Diagram (Project-Specific)

Component	Description	Purpose in E-Learning Portal
Module	Basic processing unit	Represents functional units like User Authentication, Course Management
Connection	Hierarchical relationship	Shows parent-child relationships between system modules
Data Coupling	Data transfer between modules	Transfers user credentials, course data, grade information
Control Coupling	Control information transfer	Passes authentication status, error flags, access levels
Library Module	Reusable common functions	Shared utilities like logging, validation, security functions
Conditional Call	Conditional module invocation	Role-based module access, feature toggles
Iterative Call	Repeated module invocation	Batch processing, multiple content handling

Structural Relationships and Notations

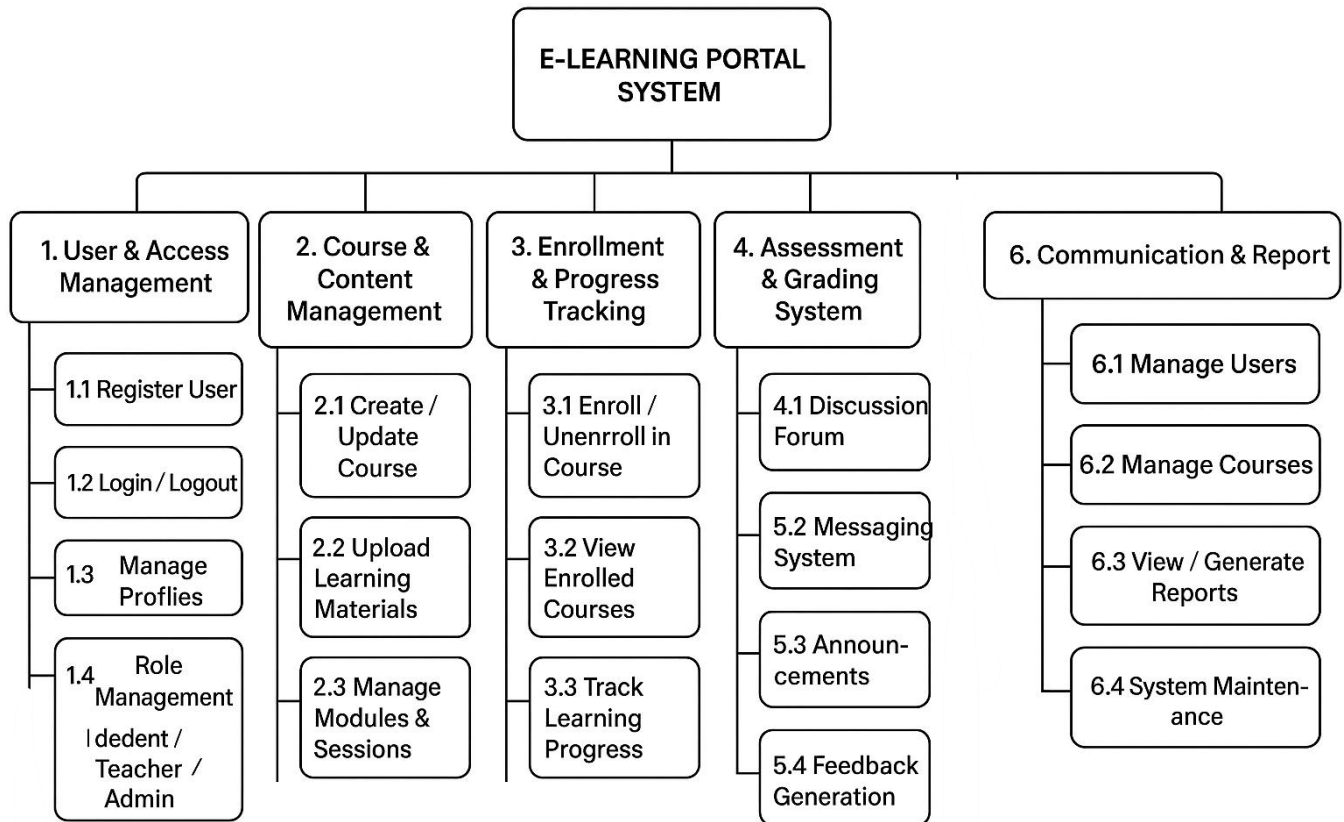
1. Hierarchical Relationships

- Parent-Child Relationship: Direct supervisory relationship between modules
- Peer Relationships: Modules at the same hierarchical level
- Cross-level Interactions: Controlled interactions between non-adjacent modules

2. Control Relationships

- Sequential Execution: Modules executed in linear sequence

- Conditional Execution: Module execution based on specific conditions
- Iterative Execution: Repeated execution of module groups
- Parallel Execution: Independent module execution paths



Applications in Software Engineering Lifecycle

1. Requirements Analysis Phase - Structure Charts help in validating functional requirements by ensuring all system functions are properly allocated to specific modules.

2. System Design Phase - They serve as the primary design artifact, guiding the architectural decisions and module interfaces.

3. Implementation Phase - Provides developers with clear module specifications and integration points.

4. Testing Phase - Facilitates unit testing, integration testing, and system testing by defining testable units.

5. Maintenance Phase - Supports impact analysis and change management through clear module dependencies.



Experiment 6

Aim: To perform the user's view analysis for the E- Learning Portal using a Use Case Diagram.

Theory and Uses of a Use Case Diagram for the E-Learning Portal

A Use Case Diagram is a core UML behavioral model that visually maps the interactions between the E-Learning Portal (the system) and its external entities (actors). Its primary function is to capture the complete set of functional requirements from the user's perspective, focusing on the goals a user achieves (the *what*), not the internal logic (the *how*). This makes it essential for validating requirements during the analysis phase.

For the E-Learning Portal, this diagram is crucial for visualizing how the three key user groups—Students, Teachers, and Administrators—interact with the platform's core modules (Course, Assessment, User Management) to streamline the academic process.

Components of the Use Case Diagram (Project-Specific)

Component	Description	Example Relevant to E-Learning Portal SRS
Actors	External entities that initiate interaction or are affected by the system.	Student, Teacher, Administrator. (Representing User Classes from SRS Section 2.3).
Use Cases	A function or goal achieved by an actor through the system.	Enroll in Course, Upload Materials, Generate Reports. (Corresponds to System Features in SRS Section 4).
System Boundary	The perimeter that defines the scope of the E-Learning Portal application.	Contains all system functionalities, separating them from external users. (Defines the scope in SRS Section 1.4).
Relationships	The connections between actors and use cases, or between use cases themselves.	Association (Teacher → Create Course), «include» (Attempt Quiz «include» Auto-Grade Quiz), «extend» (Login «extend» Forgot Password).

Uses of a Use Case Diagram (Project Application)

The Use Case Diagram is a powerful tool specifically utilized for the following functions in the E-Learning Portal project lifecycle:

1. To Define and Validate Functional Scope (SRS Compliance):

- It visually confirms that all requirements listed in SRS Section 4 (System Features) are accounted for, such as the separation of features across the Student Module, Teacher Module, and Admin Module.
- The system boundary serves as a strict visual definition of the Product Scope (SRS Section 1.4), preventing uncontrolled expansion (scope creep).

2. To Clarify User-Role Permissions (Role-Based Access):

- The diagram clearly shows which actor can initiate which use case, directly supporting the **Role-Based Access Control** constraint (SRS Section 2.5, 5.3).
- For example, it confirms that only the **Teacher** has the association with the "**Grade Assessments**" use case, while only the **Administrator** has access to "**Manage Users**."

3. To Simplify Communication for Stakeholders:

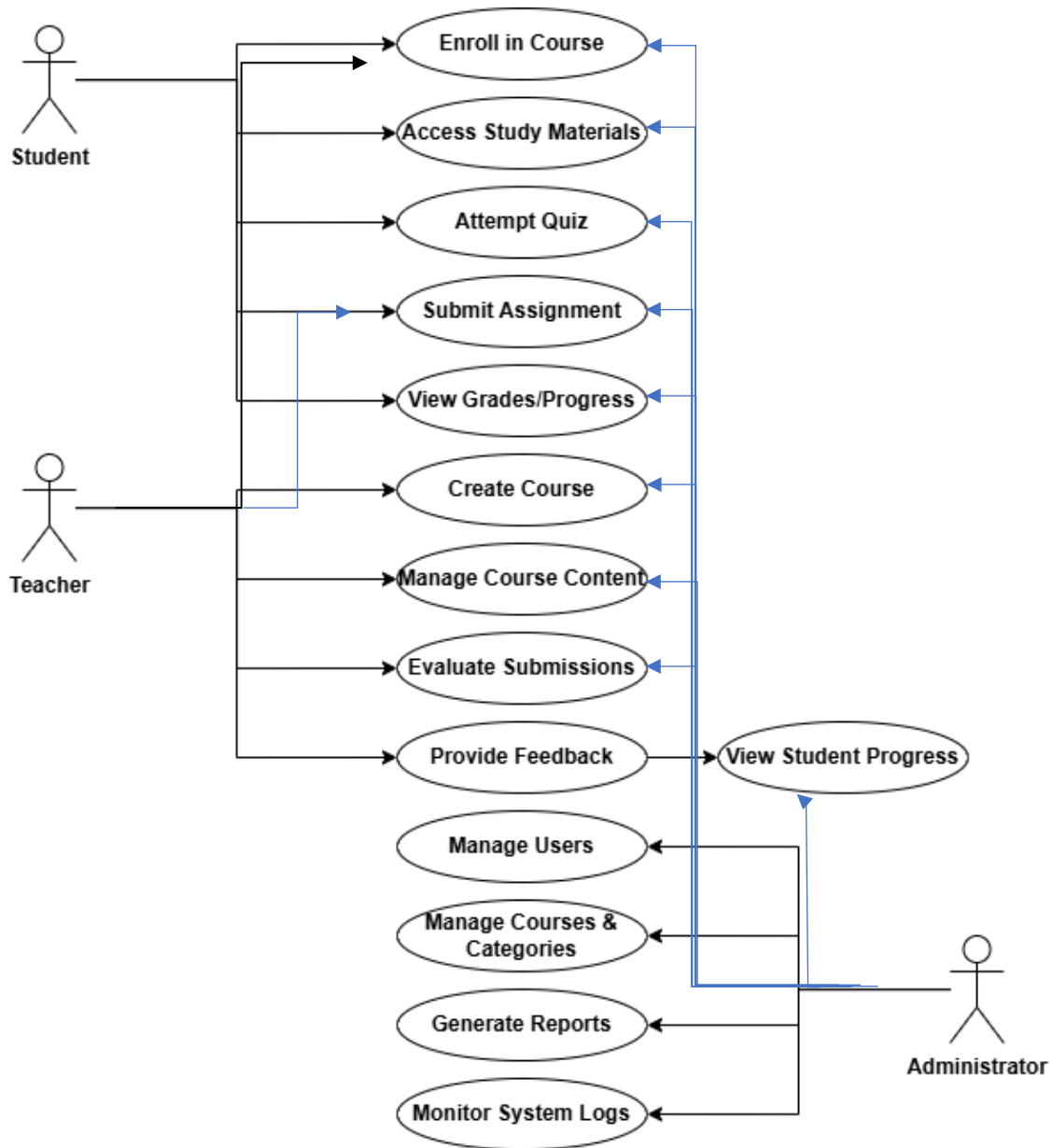
- It translates the technical requirements into a simple graphical format, bridging the communication gap between the **Developers** (who focus on implementation) and **Administrators/Teachers** (who are concerned with functionality) (SRS Section 1.3).
- It ensures a common understanding across the diverse intended audience.

4. To Structure Development and Testing Efforts:

- Each use case serves as a fundamental **unit of work** for both development and testing. For instance, the "**Attempt Quiz**" use case directly leads to the creation of test scenarios to check the **Assessment Module** (SRS Section 4.5).
- **<<include>>** and **<<extend>>** relationships, such as the one showing **Login** being **<<include>>**-d by **Authenticate User**, help the design team identify **reusable and modular components** in the backend system.

5. To Model User-Centric Goals:

- By focusing on actor goals (e.g., **Student's** goal is to "**Track Progress**," Teacher's is to "**Provide Feedback**"), the diagram ensures the final system aligns with the intended purpose of enhancing learning outcomes (SRS Section 1.4).



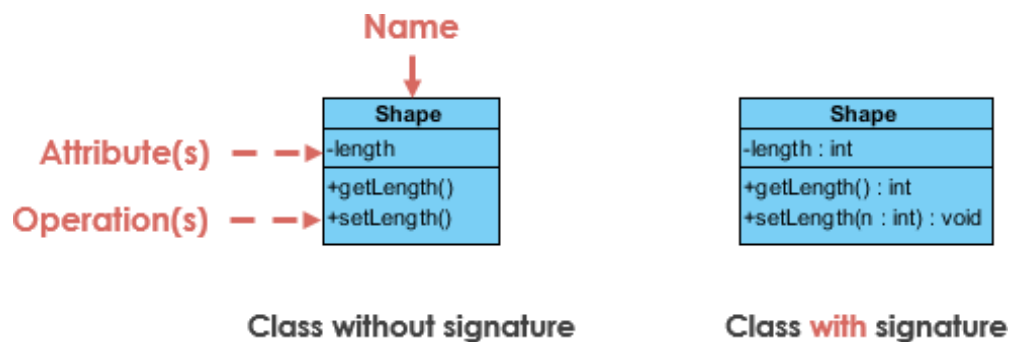
Use Case Diagram for the E-Learning Portal

Experiment 7

Aim: To draw the structural view diagram for the system: class diagram, object diagram.

UML Class Notation

A class represent a concept which encapsulates state (attributes) and behavior (operations). Each attribute has a type. Each operation has a signature. The class name is the only mandatory information.



Class Name:

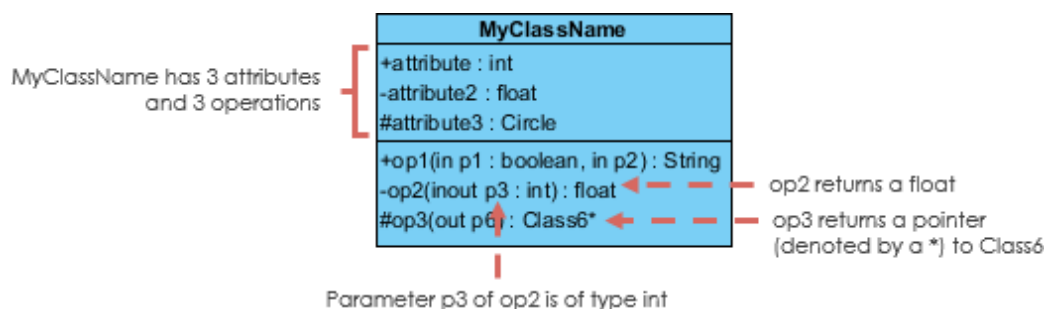
- The name of the class appears in the first partition.

Class Attributes:

- Attributes are shown in the second partition.
- The attribute type is shown after the colon.
- Attributes map onto member variables (data members) in code.

Class Operations (Methods):

- Operations are shown in the third partition. They are services the class provides.
- The return type of a method is shown after the colon at the end of the method signature.
- The return type of method parameters are shown after the colon following the parameter name. Operations map onto class methods in code

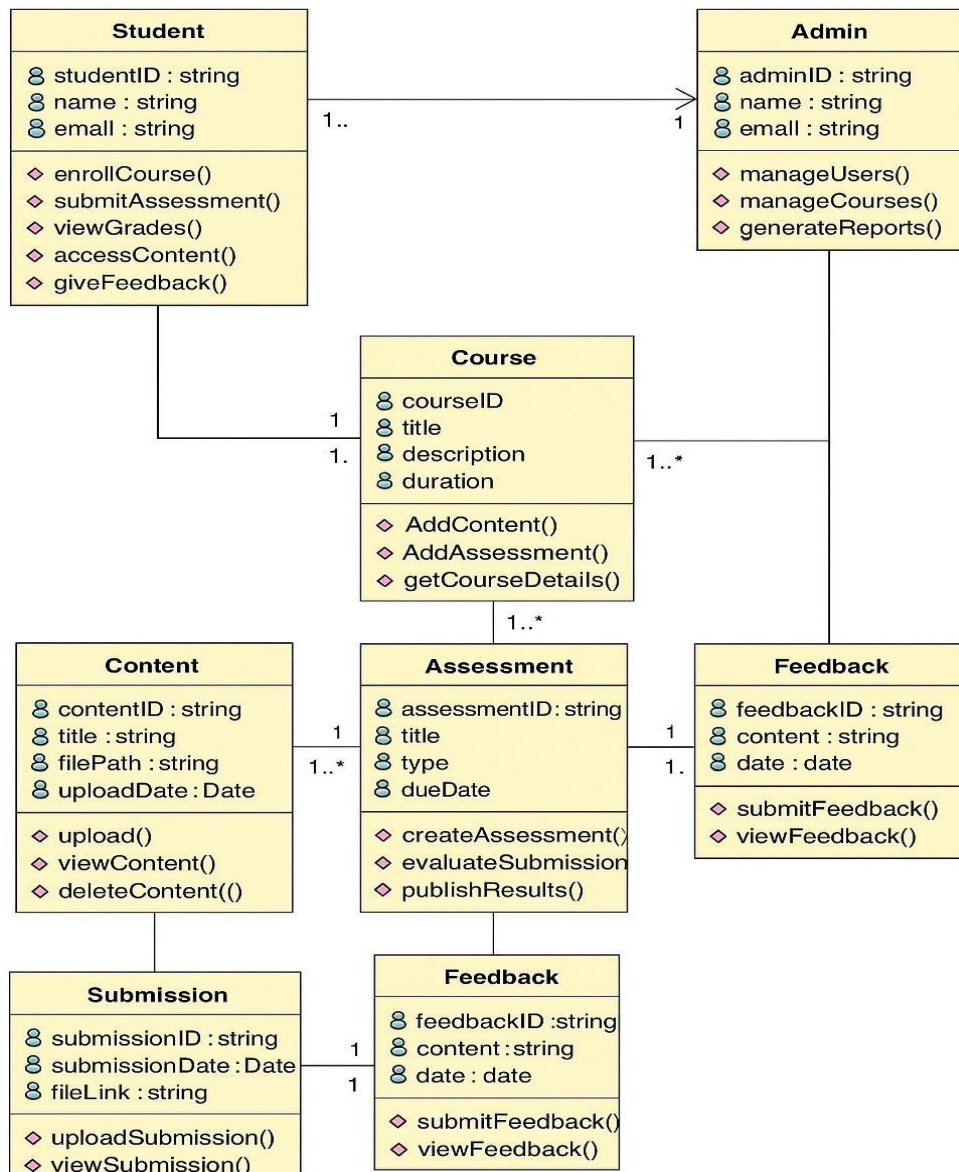


Class Diagram: -

A Class Diagram in UML (Unified Modeling Language) is a static structure diagram that shows the classes, their attributes, operations (methods), and the relationships among objects. It is one of the most commonly used diagrams in object-oriented modeling.

A diagram can be interpreted from various perspectives:

- **Conceptual:** represents the concepts in the domain
- **Specification:** focus is on the interfaces of Abstract Data Type (ADTs) in the software
- **Implementation:** describes how classes will implement their interfaces



Class Diagram for E- Learning portal

Object Diagram: -

An Object Diagram is a static structure diagram in UML (Unified Modeling Language) that represents a snapshot of instances (objects) and their relationships at a particular moment in time. It is derived from a class diagram and shows actual objects (not classes) along with values of their attributes.

Purpose

- To visualize, understand, and analyze the object-level relationships in a system.
- To test the accuracy of a class diagram by showing how instances interact at runtime.
- To capture a specific state of the system (e.g., after login, after course enrollment).
- To validate data flow and associations among system objects.

Notations Used in Object Diagram

1. Object (Instance)

- Represented by a rectangle with the object name followed by a colon and class name.
- Example:

student1: Student

The object name (student1) is optional; if omitted, it denotes an anonymous object.

2. Attributes and Values

- Shown inside the object box below the name.
- Example:

student1 : Student

name = "Sameera"

email = "sameera123@gmail.com"

3. Links (Associations between Objects)

- Represented by **solid lines** connecting two object instances.
- Example:

student1: Student ----- course1: Course

4. Multiplicity

- Shown near the link to indicate the number of participating instances.
- Example:

1..* or 0..1

5. Messages (optional)

- Can be labeled on the link to show communication (in dynamic object diagrams).

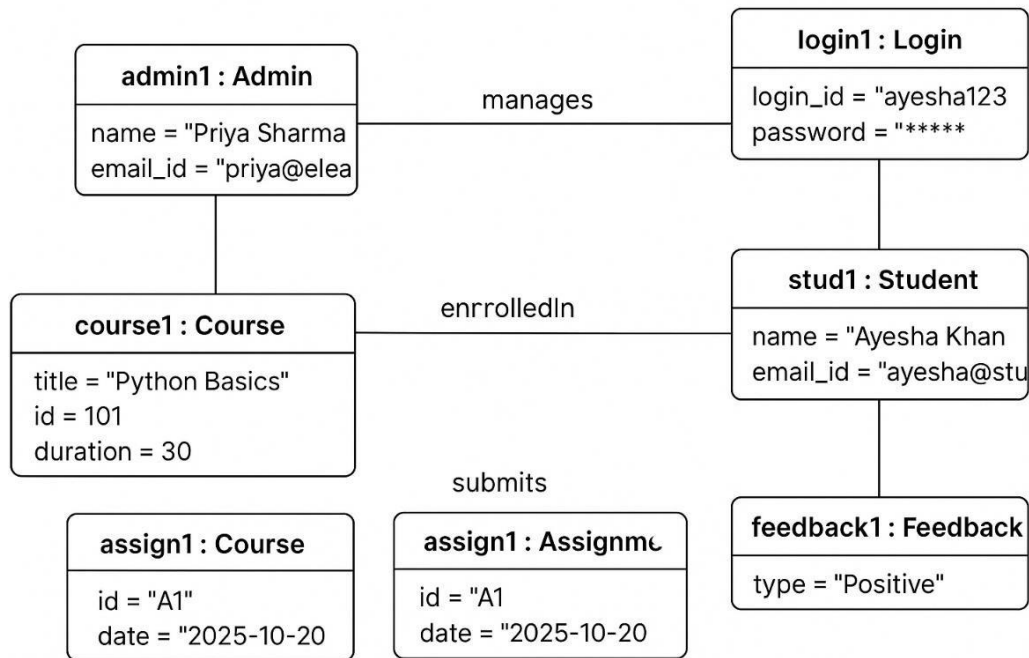
Object Diagram for E-Learning Portal

In the E-Learning Portal System, the object diagram captures a real instance of interaction among objects such as:

- student1: Student
- professor1: Professor
- admin1: Admin
- course1: Course
- assignment1: Assignment
- feedback1: Feedback
- login1: Login

Example Scenario:

- student1 logs in using login1.
- student1 is enrolled in course1.
- professor1 manages course1.
- student1 submits assignment1 under course1.
- feedback1 is linked to student1 and professor1.



Object Diagram for E- Learning portal

Conclusion:

The experiment successfully demonstrated the creation and interpretation of UML Class and Object Diagrams for an E-Learning Portal system. The class diagram illustrated the system's structure by defining various classes such as Student, Professor, Course, Assignment, and Feedback along with their attributes, methods, and relationships. The object diagram represented real-world instances of these classes and their interactions at a specific moment, such as a student enrolling in a course or submitting an assignment. This experiment helped in understanding how UML diagrams model both the static and dynamic aspects of an object-oriented system, providing a clear blueprint for software design and implementation.

Experiment 8

Aim: To draw the behavioural view diagram: state chart diagram.

State Diagram: -

A State Diagram (also known as a State Machine Diagram) in UML models the dynamic behavior of a system by showing how an object transitions between different states in response to events or actions.

It helps visualize how an object reacts to inputs, what states it can be in, and how it moves from one state to another during its lifetime.

Purpose

- To describe the **lifecycle of an object**.
- To show how an object **changes its state** depending on events.
- To represent **event-driven behavior** in reactive systems.
- To help in **system design**, especially for **control-based or real-time systems**.

Key Components

1. States:

Represent the different conditions or situations of an object during its lifetime. Example: Idle, Processing, Completed.

2. Initial State (Start State):

Denotes where the object starts. Represented by a **filled black circle (●)**.

3. Final State (End State):

Represents the end of the object's life. Shown as a **bullseye symbol (◎)**.

4. Transitions:

Arrows that show movement from one state to another triggered by an **event**.

5. Event/Trigger:

The cause that initiates a transition.

Example: submitRequest, cancelOrder.

6. **Action:**

The activity performed during a transition or in a state.

7. **Guard Condition:**

A Boolean expression that must be true for a transition to occur.

Example: [balance > 0]

State Chart Diagram for E-Learning Portal

For Student

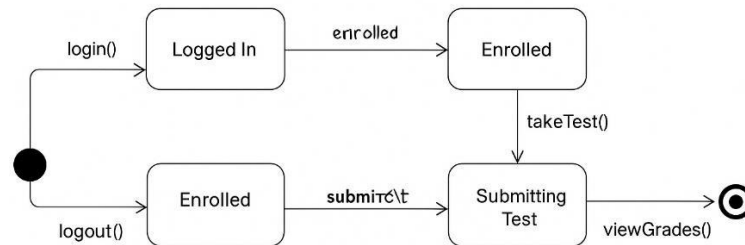
States:

1. **Idle/Not Logged In** – Initial state before login.
2. **Logged In** – Student successfully logs in.
3. **Course Browsing** – Student views available courses.
4. **Enrolled** – Student joins a selected course.
5. **Learning/Attending Lecture** – Student watches lectures or reads material.
6. **Attempting Quiz** – Student takes tests or quizzes.
7. **Submitting Assignment** – Student uploads assignments.
8. **Viewing Results** – Student checks grades or progress.
9. **Logged Out** – Final state after logout.

Transitions:

Idle → Login → Logged In → Browse Courses → Enroll → Attend Lectures → Attempt Quiz → Submit Assignment → View Result → Logout.

State Chart Diagram for Student



State chart Diagram for student

For Admin

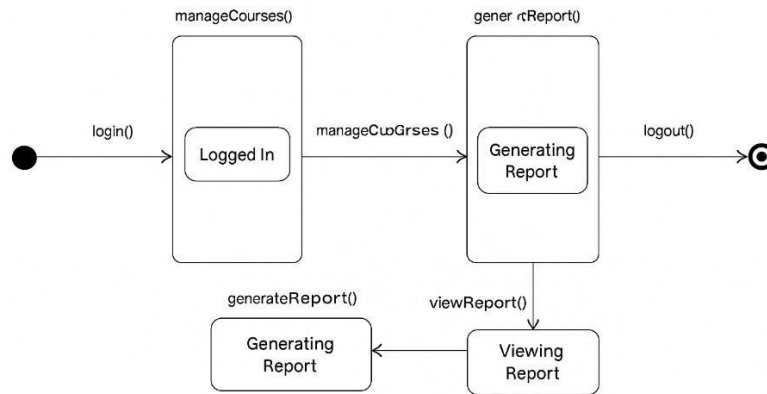
States:

1. **Idle/Not Logged In**
2. **Admin Logged In**
3. **Managing Courses** – Add, update, or delete course details.
4. **Managing Students** – Verify new registrations, manage profiles.
5. **Managing Content** – Upload lectures, quizzes, notes.
6. **Viewing Reports** – View system performance, student progress.
7. **Logged Out**

Transitions:

Idle → Login → Admin Logged In → Manage Courses/Students/Content → View Reports
→ Logout.

State Chart Diagram for Admin



State chart Diagram for Admin For

Overall E-Learning System

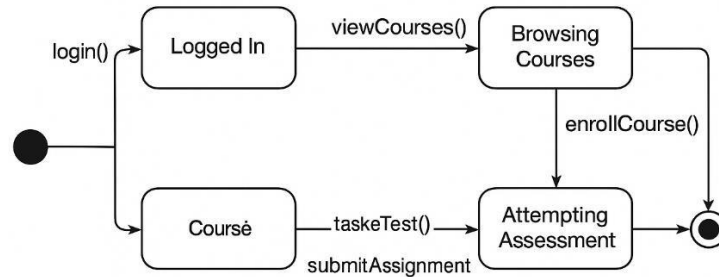
States:

1. **System Initialized** – Server starts, database connected.
2. **User Authentication** – User logs in (student/admin).
3. **User Dashboard Active** – Based on role (student/admin).
4. **Processing Requests** – Enroll, upload, quiz, etc.
5. **Updating Database** – Store results, submissions.
6. **System Idle** – Waiting for next user request.
7. **System Shutdown** – End of service.

Transitions:

Initialized → Authenticate User → Dashboard Active → Process Request → Update Database
→ Idle → Shutdown.

State Chart Diagram for E-Learning Portal



State chart Diagram for E- Learning portal

Conclusion:

The experiment successfully illustrated the creation of a State Chart Diagram to represent the behavioral aspects of the E-Learning Portal system. It demonstrated how various entities, such as students and administrators, transition between different states based on events and actions — for instance, login, course enrollment, assignment submission, and logout. Through this diagram, the dynamic flow and lifecycle of each object were effectively visualized, enhancing understanding of event-driven behavior and system responses. This experiment thus provided valuable insight into modeling real-time interactions and designing responsive systems using UML state diagrams.

Experiment 9

Aim: To draw the behavioral view diagram: activity diagram

Theory and Uses of an Activity Diagram

An Activity Diagram is a behavioral diagram in the Unified Modeling Language (UML) that represents the workflow of step-by-step actions and the flow of control in a system. It focuses on the dynamic aspects of the system, illustrating the sequence and conditions for coordinating activities. Unlike structure charts which show a static module hierarchy, activity diagrams model the operational workflows, parallel processes, and decision points, making them ideal for visualizing the flow from one activity to another.

Components of the Activity Diagram (Project-Specific)

Component	Description	Purpose in E-Learning Portal
Start Node	The initial state of the workflow.	Represents the beginning of the user login process.
Activity/Action	A step in the workflow where a task is performed.	Represents actions like "Enter User ID and Password," "Browse Courses," etc.
Control Flow	Shows the sequence and direction of the flow from one activity to another.	Connects activities in the logical order they occur.
Decision Node	A diamond symbol representing a branching point with alternate paths.	Checks conditions like "[Valid login details]" or "[Invalid login details]".
Merge Node	A diamond symbol used to bring back together multiple alternate flows.	Combines flows after a decision point (though often implied in simpler diagrams).
Fork Node	Splits a single flow into multiple concurrent (parallel) flows.	After login, different roles (Admin, Student, Professor) can perform their respective activities simultaneously.
Join Node	Synchronizes multiple concurrent flows back into a single flow.	Ensures that all necessary parallel actions are complete before moving to a common step, like "Logout".
End Node	The final state of the workflow.	Represents the conclusion of the session, "Logout User".

Structural Relationships and Notations

1. Sequential Flow

- The primary flow of control from one activity to the next in a linear sequence.

2. Conditional Flow

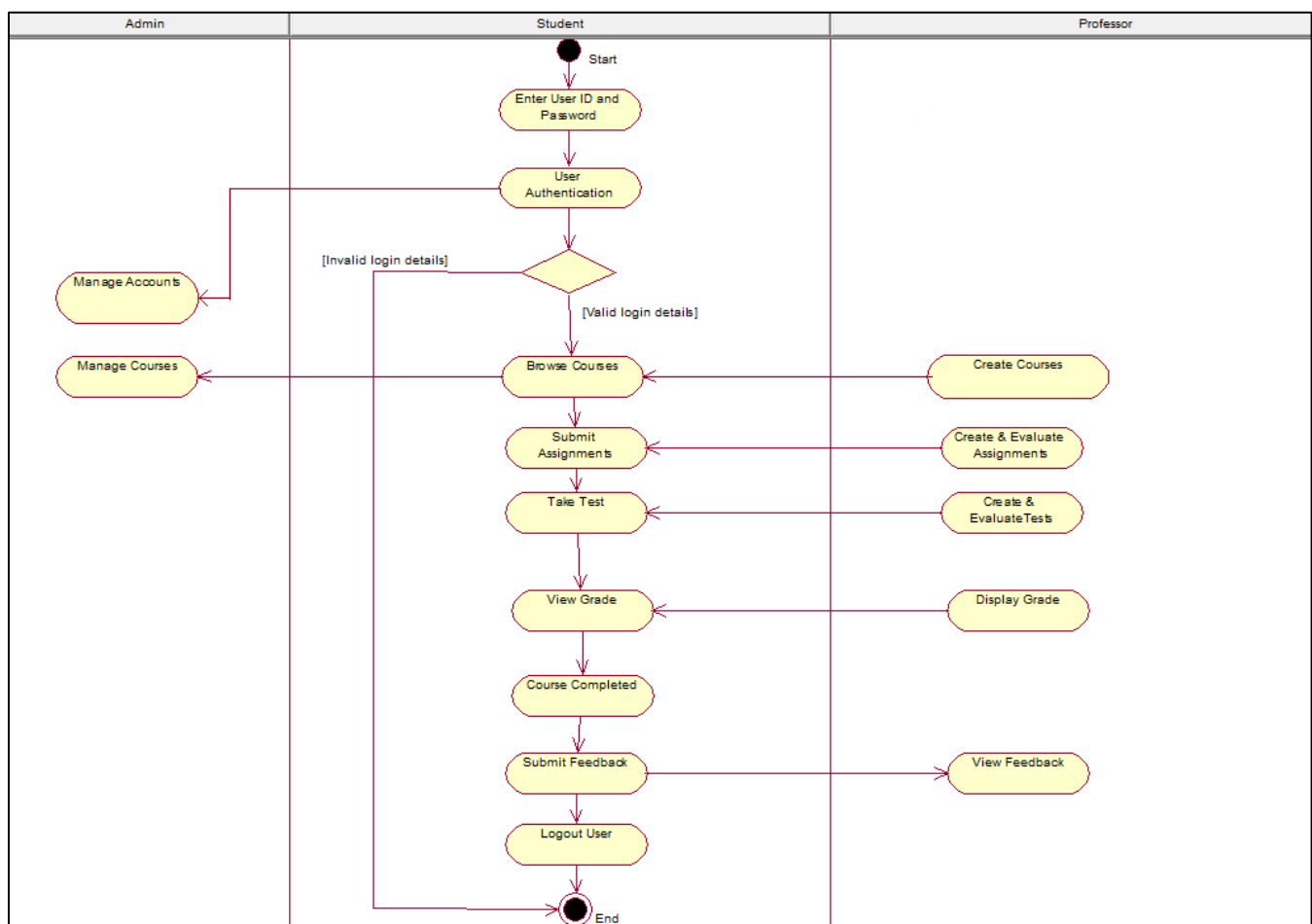
- Represented by Decision Nodes, the flow diverges based on guard conditions (e.g., [Valid login details]).

3. Concurrent Flow

- Represented by Fork and Join Nodes, allowing multiple activities to occur in parallel (e.g., a student browsing courses while a professor creates an assignment).

4. Exception Flow

- An alternate path taken when an error or exception condition is met (e.g., the [Invalid login details] flow leading back to the start).



Applications in Software Engineering Lifecycle

1. Requirements Analysis Phase - Activity Diagrams help in understanding and validating user stories and use cases by visualizing the step-by-step workflow from the user's perspective.

2. System Design Phase - They model the logic of complex operations, business processes, and the concurrency within the system, which is crucial for designing efficient systems.

3. Implementation Phase - Provides developers with a clear map of the control flow, conditional logic, and parallel processes that need to be coded.

4. Testing Phase - Facilitates the creation of test cases by clearly outlining all possible paths, scenarios, and decision outcomes that must be tested.

5. Maintenance Phase - Helps in understanding the impact of a change in a business process by showing how activities are linked and which parts of the system are affected by a modification.

Conclusion:

The experiment effectively demonstrated the construction and interpretation of an Activity Diagram for the E-Learning Portal system. The diagram showcased the step-by-step workflow of activities, including login, browsing courses, enrolling, attending lectures, and logout. It provided a clear visualization of control flow, decision points, and concurrent actions for different user roles such as students, professors, and admins. Through this, the dynamic behavior of the system was well-understood, aiding in process optimization and error handling. Overall, this experiment emphasized the importance of activity diagrams in analyzing, designing, and improving the operational flow of a system within the software development lifecycle.

Experiment 10

Aim: To perform the behavioral view diagram for the E learning portal: sequence diagram for Student and Professor

Theory and Uses of a Sequence Diagram

A Sequence Diagram is a behavioral diagram in the Unified Modeling Language (UML) that illustrates how objects interact in a time-ordered sequence. It focuses on the messaging between different system components (objects) to accomplish a specific functionality, highlighting the chronological order of these interactions. This diagram is crucial for understanding the dynamic behavior of a system during the execution of a use case, making it particularly valuable for modeling user-system interactions and internal component communications.

Components of the Sequence Diagram (Project-Specific)

Component	Description	Purpose in E-Learning Portal
Lifeline	Represents an object participating in the interaction over time.	Student, Professor, HomeScreen, Course, Database
Activation Bar	A thin rectangle on a lifeline showing the period during which the object is performing an operation.	Shows when an object is active while processing a request.
Message	A communication between lifelines that conveys information.	Validate User(), Select Course(), Create Course Material()
Return Message	A dashed arrow showing the response from a previous message.	Returning Success after validation or returning data like Available Courses.
Self-Message	A message an object sends to itself.	Represents an object performing an internal computation.
Found Message	A message from an unknown or unspecified sender.	Represents an external trigger, like a user initiating the login process.
Guard Condition	A condition in square brackets [] that must be true for a message to be sent.	[Invalid User()] leading to an alternative flow.

Structural Relationships and Notations

1. Sequential Messaging

- Messages are processed in a strict chronological order from top to bottom.

2. Synchronous Communication

- Represented by a solid arrowhead. The sender waits for a response before proceeding (e.g., Validate User()).

3. Asynchronous Communication

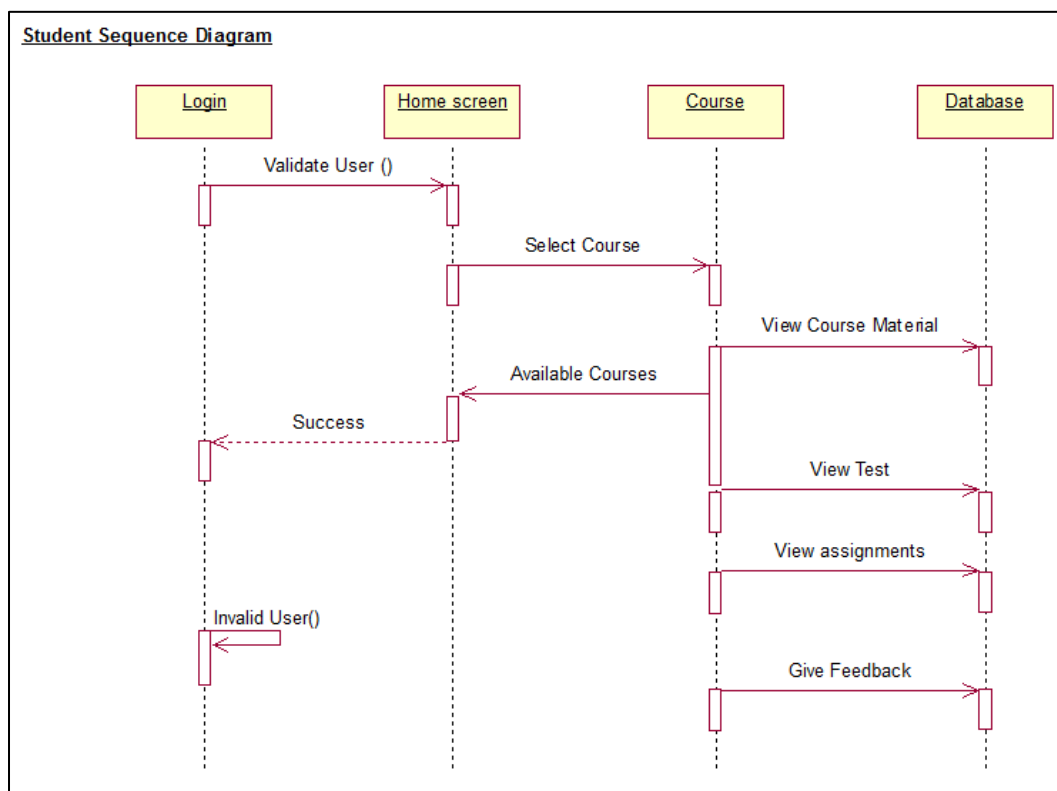
- Represented by a lined arrowhead. The sender does not wait for a response and continues immediately.

4. Object Creation and Destruction

- Creation is shown by a message pointing to the object's lifeline with the «create» keyword. Destruction is shown with a X at the end of a lifeline.

I. Student Sequence Diagram

This diagram models the interaction flow for a student user, focusing on consuming educational content and receiving evaluations.



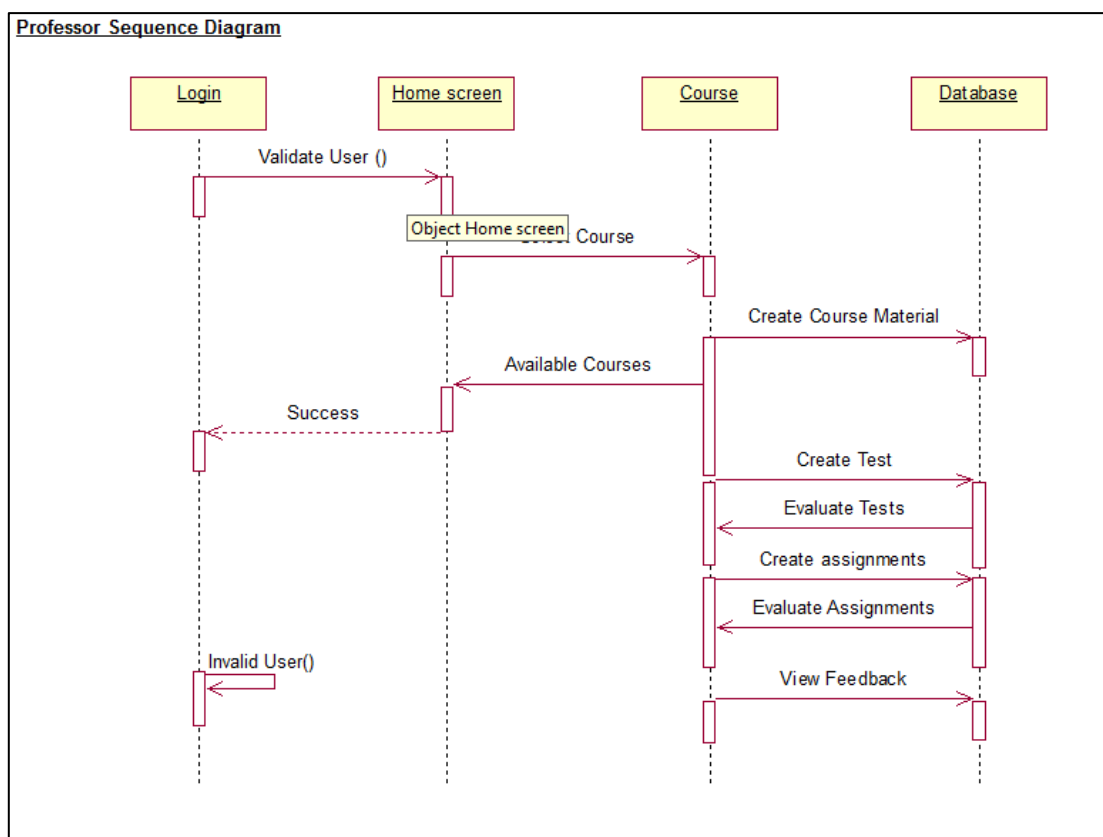
Key Interactions for Student:

- **Authentication:** The sequence begins with user validation (Validate User()).

- **Course Selection:** The student selects and views available courses (Select Course, View Course Material).
- **Academic Activities:** The student interacts with tests and assignments (View Test, View assignments).
- **Feedback:** The session may conclude with the student providing feedback (Give Feedback).

2. Professor Sequence Diagram

This diagram models the interaction flow for a professor user, focusing on content creation and student evaluation.



Key Interactions for Professor:

- **Authentication:** The sequence begins with user validation (Validate User()).
- **Course Management:** The professor can create and manage course content (Create Course Material, Create Test, Create assignments).
- **Evaluation:** The professor evaluates student submissions (Evaluate Tests, Evaluate Assignments).

- **Feedback Review:** The professor views feedback received from students (View Feedback).

Applications in Software Engineering Lifecycle

1. Requirements Analysis Phase - Sequence Diagrams help in detailing use case scenarios, making complex interactions between different user roles (Student, Professor) and the system clear and unambiguous.

2. System Design Phase - They are instrumental in designing the API and method calls between different software objects or microservices, ensuring a logical flow of control for distinct user functionalities.

3. Implementation Phase - Provides developers with a near-direct blueprint for writing method calls and structuring object interactions in code, tailored to specific actor permissions and workflows.

4. Testing Phase - Serves as a basis for creating integration test cases by defining the expected sequence of interactions that must occur between components for both student and professor operations.

5. Maintenance Phase - Helps in debugging and impact analysis by showing how a change in one object's interface can affect the sequence of interactions for multiple user roles within the system.

Conclusion:

The experiment successfully illustrated the development of Sequence Diagrams for both Student and Professor modules in the E-Learning Portal. These diagrams effectively represented the chronological flow of messages and interactions among system objects such as Student, Professor, Course, and Database. They helped visualize how authentication, course selection, content creation, assignment evaluation, and feedback processes occur in a time-ordered manner. Through this experiment, the dynamic behavior and communication logic of the system were clearly understood, making it easier to design, implement, and test the system's functional flow. Overall, this enhanced understanding of real-time object interactions in UML-based modeling.